M20 CI/CD & Production Security

Methodology Advanced

Using Claude Code in production with the right security guarantees

The `--dangerously-skip-permissions` flag

This flag bypasses permission prompts. Headless tasks can instead use pre-approved tools and a restricted tool set. But it also removes the last line of defense against destructive actions. Its use is only acceptable inside an isolated ephemeral container, never on a shared persistent machine.

```
# Acceptable in CI (ephemeral container)
claude --dangerously-skip-permissions \
  -p "Run tests and fix failing ones"

# Unacceptable (shared machine, open network access)
claude --dangerously-skip-permissions  # ❌
```

The rule: the flag widens the trust granted to Claude, so the container perimeter must compensate by reducing what Claude can reach.

Recommended environment

A disposable Docker container created at job start and destroyed at the end. The repository is mounted as a volume, dependencies installed inside, and no sensitive files from the host machine are accessible.

```
jobs:
  agent-review:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@v4
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${{ secrets.ANTHROPIC_API_KEY }}
          prompt: "Review the checked-out source and summarize defects."
          claude_args: '--tools "Read,Grep,Glob" --allowedTools
"Read,Grep,Glob"'
```

This example grants source-reading access. Credentials, runner isolation and workflow triggers need a separate review. MicroVMs add a separate kernel boundary; no virtualization mechanism eliminates every escape or data-access risk.

Strict tool whitelist

Even in bypass mode, it is possible to limit available tools. A review agent only needs `Read`, `Glob`, `Grep`. An implementation agent can have `Edit` on targeted paths, but rarely `Bash` without constraints.

Agent role	Allowed tools
Code review	Read, Glob, Grep
Test fixing	Read, Edit, Bash(test runner)
Security analysis	Read, Glob, Grep
Implementation	Read, Edit, Write, Bash

Use `--tools` to restrict which tools are available and `--allowedTools` to pre-approve permitted calls. Pass these through `claude_args` in the official action.

Secrets in CI

API keys and tokens must never appear in the workflow YAML file. Use GitHub secrets (Settings → Secrets and variables → Actions) and inject them as environment variables.

```
env:
  ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
  # Never:
  # ANTHROPIC_API_KEY: sk-ant-xxxxx ❌
```

Recommended rotation: change keys every 90 days and audit access via GitHub logs.

Network: restricted access

Never run Claude in bypass mode with unrestricted network access. Configure an explicit network allowlist for legitimate domains:

```
api.anthropic.com   # Model
*.npmjs.org         # Packages
github.com          # Git operations
```

Everything else deny by default. This restricts destinations, but an allowed service can still receive sensitive data. Review credentials, request scope and the data accessible to the agent.

Safe autonomy pattern

The safest pattern for production: Claude proposes, a human approves. The agent opens a PR with its changes, the test pipeline validates, and a human reviewer approves before the merge.

```
Agent → branch → PR → CI tests → human review → merge
```

Do not automate the final merge, even if all tests pass. Keep a human in the loop for changes to main.

Anti-patterns to avoid

Anti-pattern	Risk
Skip permissions on host machine	Unbounded filesystem/network access
Mounting `/home` in the container	Exposure of SSH keys, credentials
Network allowlist `*`	Possible exfiltration
Automatic merge without review	Undetected regression